ByteMart Digital

THE COMPLETE GUIDE

Java

From Fundamentals to Professional Practice

A complete, example-driven guide to modern Java — from the JVM, syntax, and OOP to collections, streams, generics, concurrency, memory, and professional practice.

30	200+	14	8
Chapters	Code Examples	Visual Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Java: The Complete Guide

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided "as is".

Trademarks
Java is a registered trademark of Oracle and/or its affiliates. All other product names, logos, and brands are the property of their respective owners and are used for identification purposes only.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. What Is Java? 4
2. Variables & Data Types 8
3. Operators & Expressions 11
4. Control Flow 14
5. Classes & Objects 17
6. Methods & Constructors 20
7. Inheritance 23
8. Polymorphism, Abstraction & Encapsulation 26
9. Interfaces & Abstract Classes 29
10. Packages & Access Modifiers 32
11. Static, Final & Nested Classes 35
12. Enums & Records 38
13. Strings & Text 41
14. Arrays 44
15. Exceptions & Error Handling 47
16. Generics 50
17. The Collections Framework 53
18. Iterators & Comparators 56
19. Lambdas & Functional Interfaces 59
20. The Streams API 62
21. Optional & Null Safety 65
22. Concurrency & Threads 68
23. Concurrency Utilities 71
24. Files & I/O 74
25. Memory & Garbage Collection 77
26. equals, hashCode & Object Contracts 80
27. Annotations & Reflection 83
28. Build Tools & the Ecosystem 86
29. Testing with JUnit 89
30. Best Practices & Clean Code 92

MODULE 1 · JAVA FOUNDATIONS

CHAPTER 01

What Is Java?

A platform-independent, object-oriented language — how it compiles, runs, and why WORA matters.

01

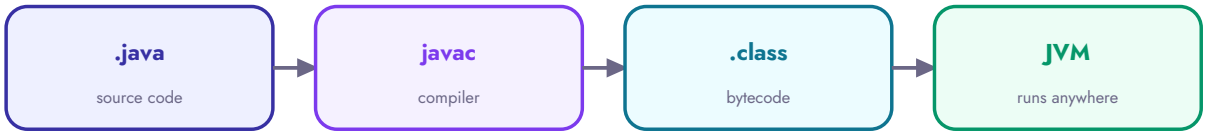
Java is a general-purpose, object-oriented programming language designed to run anywhere. Its promise — **write once, run anywhere** — comes from compiling to platform-neutral bytecode that any Java Virtual Machine can execute. Decades on, it powers banks, Android apps, and enterprise back-ends worldwide.

> **Java**
> A statically typed, object-oriented, compiled language that runs on the Java Virtual Machine (JVM), giving the same program identical behaviour across operating systems.

Compile once, run anywhere

You write `.java` source, compile it with `javac` into `.class` bytecode, and the JVM runs that bytecode on any platform. The bytecode never changes — only the JVM differs per OS.

How Java Runs: Write Once, Run Anywhere



The same bytecode runs on any OS with a JVM

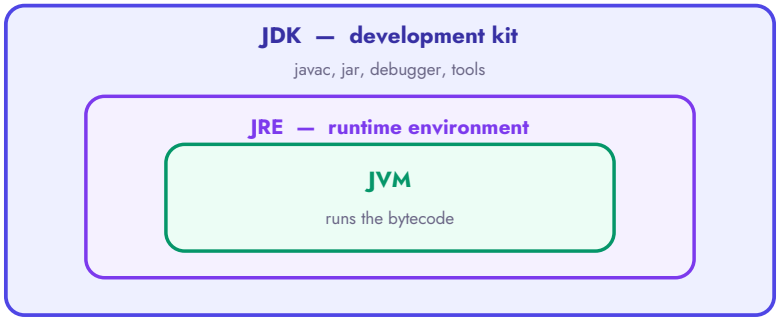
Source is compiled to bytecode once; any JVM runs that same bytecode.

JDK, JRE, and JVM

These three are often confused. The **JVM** executes bytecode. The **JRE** is the JVM plus the standard class libraries. The **JDK** is the JRE plus development tools like the compiler — what you install to write Java.

© 2026 ByteMart Digital

JDK, JRE, and JVM



The JDK contains the JRE, which contains the JVM

The JDK contains the JRE, which contains the JVM.

Your first program

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, ByteMart!");
    }
}
```

OUTPUT

```
Hello, ByteMart!
```

Every application starts at `main`. Compile with `javac Hello.java` and run with `java Hello`.

What defines Java

- **Object-oriented** — code is organised into classes and objects.
- **Statically typed** — types are checked at compile time.
- **Platform-independent** — bytecode runs on any JVM.
- **Automatic memory management** — the garbage collector frees unused objects.
- **Backward compatible** — old code keeps running on new versions.

TIP

Use a recent long-term-support release (such as Java 17 or 21). LTS versions get years of updates and are what most companies run in production.

One language, every platform

Java's staying power comes from the JVM: compile to bytecode once, and the same program runs on servers, desktops, and Android alike. That portability is still its defining strength.

© 2026 ByteMart Digital

© 2026 ByteMart Digital

MODULE 1 · JAVA FOUNDATIONS

CHAPTER 02

Variables & Data Types

Declaring variables, Java's eight primitive types, and the difference between value and reference.

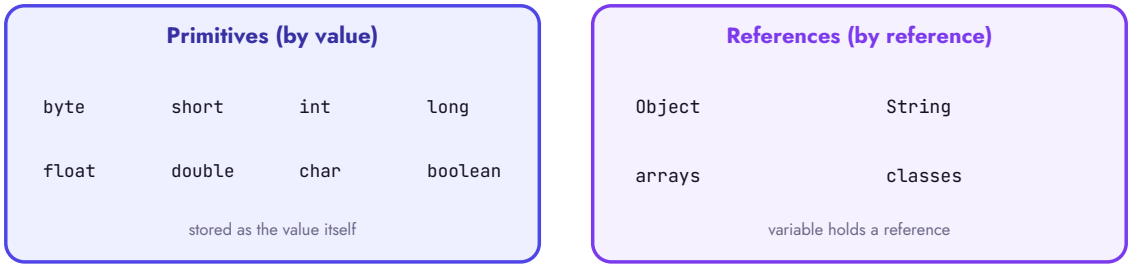
Every variable in Java has a declared type, fixed at compile time. Types fall into two groups: **primitives**, which hold raw values, and **reference types**, whose variables point to objects. Knowing the difference prevents a whole class of bugs.

The eight primitives

Type	Size	Example
int	32-bit	42
long	64-bit	10000000000L
double	64-bit	3.14
float	32-bit	3.14f
boolean	1-bit	true
char	16-bit	'A'

The other two primitives are `byte` (8-bit) and `short` (16-bit), used mainly for memory-tight or low-level code.

Primitive vs Reference Types



Primitives copy the value; references copy the pointer

Primitives store the value; reference variables store a pointer to an object.

Declaring variables

```java
int count = 0;
final double PI = 3.14159;   // final = cannot be reassigned
String name = "ByteMart";    // reference type
var total = count + 10;      // 'var' infers the type (Java 10+)
```

Value vs reference

Assigning a primitive copies its **value**. Assigning a reference copies the **reference** — both variables then point to the same object, so changes through one are visible through the other.

```java
int a = 10, b = a;
b = 20;
System.out.println(a);   // 10  (primitive copied)

int[] x = {1, 2}, y = x;
y[0] = 99;
System.out.println(x[0]);   // 99  (same array)
```

OUTPUT

```
10
99
```

Wrapper classes

Each primitive has an object **wrapper** (`int` to `Integer`, `double` to `Double`). Java auto-converts between them — **autoboxing** and **unboxing** — which is needed because collections store objects, not primitives.

WATCH OUT

Comparing wrapper objects with `==` compares references, not values. Use `.equals()` for wrappers, and prefer primitives where you can.

Know value vs reference

Whether a variable holds a value or a reference changes how it is copied, compared, and passed to methods — the foundation for understanding much of Java's behaviour.

MODULE 1 · JAVA FOUNDATIONS

CHAPTER 03

Operators & Expressions

Arithmetic, comparison, and logical operators — plus integer division and equality gotchas.

Operators combine values into expressions. Most behave as you expect, but a few corners — integer division, operator precedence, and reference equality — trip up both beginners and interview candidates.

The operator families

- **Arithmetic** — `+ - * / %`
- **Comparison** — `== != < > <= >=`
- **Logical** — `&& || !`
- **Assignment** — `= += -= *= /=`
- **Increment / decrement** — `++ --`

Integer division

Dividing two integers gives an integer — the fractional part is discarded. Use a floating-point type to keep the decimals.

```java
System.out.println(7 / 2);      // 3   (integer division)
System.out.println(7 % 2);      // 1   (remainder)
System.out.println(7.0 / 2);    // 3.5 (one operand is double)
```

OUTPUT
```
3
1
3.5
```

Equality: == vs equals()

For primitives, `==` compares values. For objects, `==` compares **references** (are they the same object?), while `.equals()` compares **contents**.

```java
String a = new String("hi");
String b = new String("hi");
System.out.println(a == b);       // false (different objects)
System.out.println(a.equals(b));  // true  (same content)
```

OUTPUT
```
false
true
```

Short-circuit logic

&& and || short-circuit: the right side is skipped when the result is already decided. This lets you guard against errors, like a null check before a method call.

```java
if (user != null && user.isActive()) {
    // isActive() only runs if user is non-null
}
```

WATCH OUT

Always use `.equals()` to compare String and other object contents. Using == on strings sometimes appears to work due to the string pool, then fails unpredictably.

Mind the corners

Integer division truncates, and == on objects compares identity, not content. These two rules cause a surprising share of Java bugs — internalise them early.

MODULE 1 · JAVA FOUNDATIONS

CHAPTER 04

Control Flow

Directing program flow with conditionals, loops, and the modern switch expression.

Control-flow statements decide which code runs and how often. Java offers the familiar `if`, `switch`, and loops — plus a modern switch expression that is safer and more concise than the classic form.

Conditionals

```java
int score = 82;
if (score >= 90)      grade = 'A';
else if (score >= 80) grade = 'B';
else                  grade = 'C';

// ternary: concise either/or
String status = score >= 40 ? "pass" : "fail";
```

The switch expression

Modern `switch` (Java 14+) uses arrow syntax, returns a value, and needs no `break` — eliminating the classic fall-through bug.

```java
String day = switch (dayNum) {
    case 1, 7 -> "Weekend";
    case 2, 3, 4, 5, 6 -> "Weekday";
    default -> "Invalid";
};
```

Loops

Loop	Use when
for	count is known
while	condition-controlled
do-while	run at least once
for-each	iterate a collection or array

```java
for (int i = 0; i < 3; i++) System.out.print(i);   // 012

for (String name : names) {   // for-each: cleanest for collections
    System.out.println(name);
}
```

break and continue

`break` exits a loop early; `continue` skips to the next iteration. Use them to keep loop bodies flat and readable.

```java
for (int n : numbers) {
    if (n < 0) continue;   // skip negatives
    if (n > 100) break;    // stop past 100
    process(n);
}
```

TIP

Prefer the **for-each** loop when you do not need the index — it is clearer and avoids off-by-one errors. Prefer the arrow-style `switch` for its safety and readability.

Flow with intent

Choose the construct that expresses your intent: for-each for iteration, the switch expression for multi-way branching, and early `break`/`continue` to keep loops readable.

© 2026 ByteMart Digital

MODULE 2 · OBJECT-ORIENTED JAVA

CHAPTER 05

Classes & Objects

Defining a class as a blueprint and creating objects from it — the heart of Java.

Java is built around classes and objects. A **class** is a blueprint that defines fields (state) and methods (behaviour); an **object** is a concrete instance created from that blueprint. Almost everything you write in Java lives inside a class.

Class
A template defining the fields and methods shared by a kind of object. An object is a specific instance of a class, with its own copy of the fields.

Defining a class

java

```java
public class Car {
    String model;      // field (instance state)
    int speed;

    void accelerate(int by) {   // method (behaviour)
        speed += by;
    }
}
```

Creating objects

The `new` keyword creates an object on the heap and returns a reference to it. Each object has its own independent copy of the fields.

java

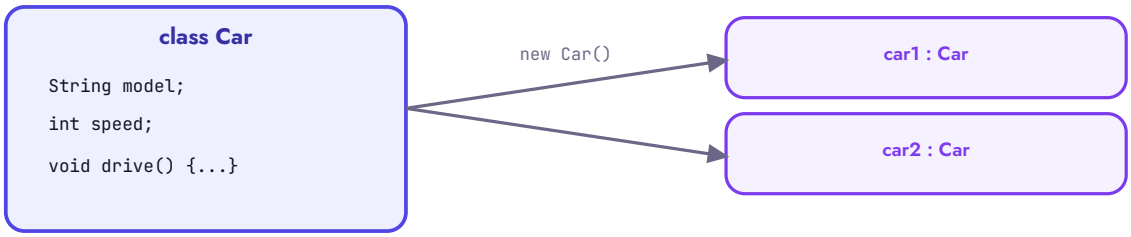
```java
Car car1 = new Car();
car1.model = "Nano";
car1.accelerate(20);
System.out.println(car1.speed);   // 20

Car car2 = new Car();             // independent object
System.out.println(car2.speed);   // 0
```

OUTPUT

```
20
0
```

Class Is a Blueprint; Objects Are Instances



One class, many independent objects with their own state

One class is a blueprint; each object is an independent instance with its own state.

Fields, methods, and this

Inside a method, `this` refers to the current object. It is used to distinguish a field from a parameter of the same name, and to pass the current object around.

```java
public class Account {
    double balance;
    void deposit(double balance) {
        this.balance += balance;   // this.balance = the field
    }
}
```

TIP

Give classes noun names (`Invoice`, `Customer`) and methods verb names (`calculateTotal`, `sendEmail`). Clear naming is half of good object design.

Blueprint and instance

A class describes what a kind of object knows and can do; objects are the live instances that hold real data. Modelling your problem as classes and objects is the essence of object-oriented Java.

MODULE 2 · OBJECT-ORIENTED JAVA

CHAPTER 06

Methods & Constructors

Defining behaviour with methods, initialising objects with constructors, and overloading both.

Methods define what an object can do; constructors define how an object is created and initialised. Java lets you provide several versions of each through **overloading**, giving flexible, readable APIs.

Method anatomy

java

```
// modifier  return-type  name(parameters)
public int add(int a, int b) {
    return a + b;
}

public void greet(String name) {   // void returns nothing
    System.out.println("Hi, " + name);
}
```

Constructors

A constructor has the same name as the class and no return type. It runs when you call `new`, setting up the object's initial state. If you write none, Java provides a default no-argument constructor.

java

```
public class Point {
    int x, y;
    public Point(int x, int y) {   // constructor
        this.x = x;
        this.y = y;
    }
}
Point p = new Point(3, 4);
```

Overloading

Overloading means several methods (or constructors) share a name but differ in their parameter lists. Java picks the right one by the arguments you pass.

java

```
public class Printer {
    void print(String s) { System.out.println(s); }
    void print(int n)    { System.out.println(n); }
    void print(String s, int times) {
        for (int i = 0; i < times; i++) print(s);
    }
}
```

© 2026 ByteMart Digital

Constructor chaining

One constructor can call another with `this(...)`, avoiding duplicated setup. This keeps a single source of initialisation logic.

java

```
public class Box {
    int w, h;
    public Box() { this(1, 1); }        // delegates
    public Box(int w, int h) { this.w = w; this.h = h; }
}
```

WATCH OUT

Overloaded methods must differ in parameter types or count — not just return type. Two methods differing only by return type will not compile.

Behaviour and creation

Methods give objects behaviour; constructors give them a valid starting state. Overloading lets one clear name serve several call shapes, and chaining keeps initialisation logic in one place.

MODULE 2 · OBJECT-ORIENTED JAVA

CHAPTER 07

Inheritance

Reusing and specialising code with the extends keyword and the super reference.

Inheritance lets a class build on another, reusing its fields and methods and adding or changing behaviour. It models an **is-a** relationship — a `Dog` is an `Animal` — and is a core tool for code reuse in Java.

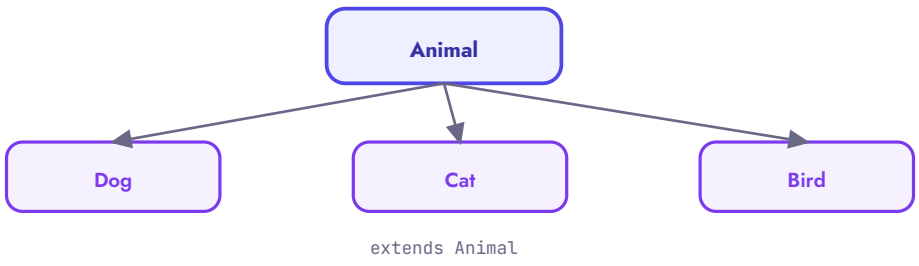
> **Inheritance**
> A mechanism where a subclass extends a superclass, inheriting its accessible fields and methods and optionally overriding or extending them.

extends

```java
class Animal {
    void eat() { System.out.println("eating"); }
}

class Dog extends Animal {   // Dog is-a Animal
    void bark() { System.out.println("woof"); }
}

Dog d = new Dog();
d.eat();   // inherited
d.bark();  // own method
```



Subclasses inherit from a superclass, forming an is-a hierarchy.

Overriding and super

A subclass can **override** an inherited method to change its behaviour. `super` calls the superclass version, and `@Override` asks the compiler to verify you really are overriding.

```java
class Cat extends Animal {
    @Override
    void eat() {
        super.eat();                  // call Animal.eat()
        System.out.println("...daintily");
    }
}
```

Constructors and super()

A subclass constructor implicitly calls the superclass's no-arg constructor first. To pass arguments up, call `super(...)` explicitly as the first statement.

```java
class Vehicle {
    Vehicle(String name) { /* ... */ }
}
class Bike extends Vehicle {
    Bike() { super("bike"); }   // must be first
}
```

The Object root

Every class implicitly extends `Object`, inheriting methods like `toString()`, `equals()`, and `hashCode()` — which you often override.

WATCH OUT

Java supports single inheritance only — a class extends at most one superclass. To combine multiple behaviours, use interfaces (covered next module).

Reuse through is-a

Inheritance shares code down a hierarchy and lets subclasses specialise via overriding. Use it for genuine is-a relationships; for everything else, prefer composition.

© 2026 ByteMart Digital

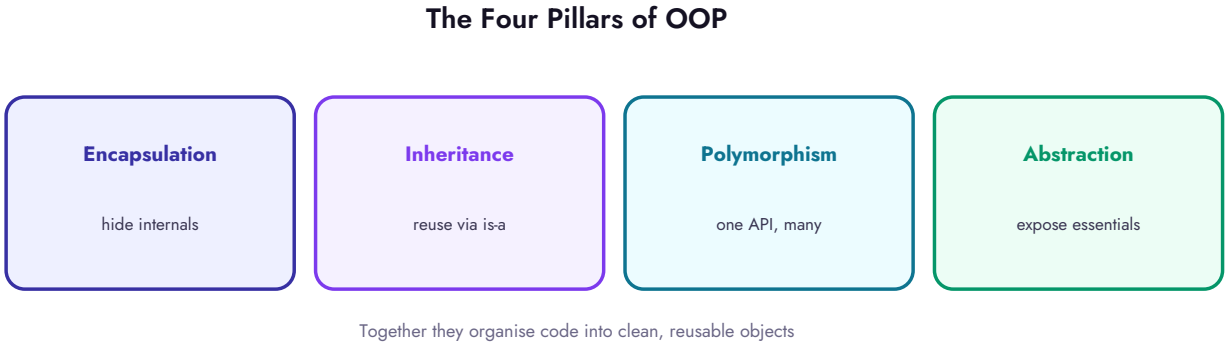
MODULE 2 · OBJECT-ORIENTED JAVA

CHAPTER 08

Polymorphism, Abstraction & Encapsulation

The other three OOP pillars — one interface with many forms, hiding detail, and protecting state.

With inheritance, these complete the four pillars of object-oriented design. **Polymorphism** lets one reference behave as many types; **abstraction** exposes only what matters; **encapsulation** guards an object's internal state.

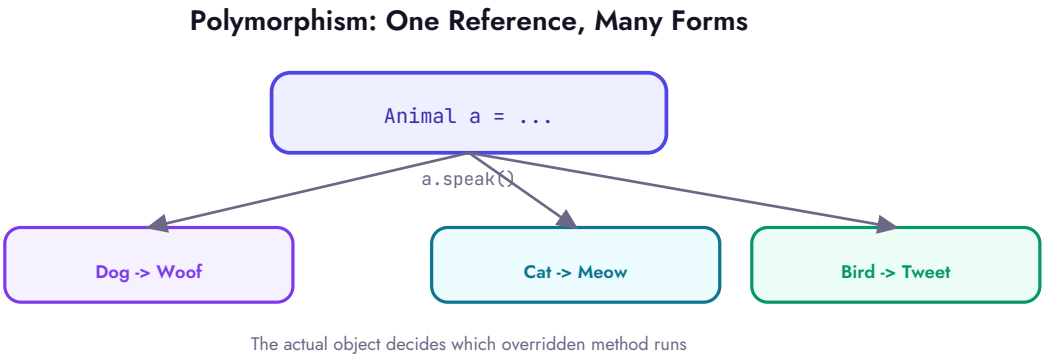


Encapsulation, inheritance, polymorphism, and abstraction — the four pillars of OOP.

Polymorphism

A superclass reference can point to any subclass object, and the **actual** object decides which overridden method runs. This lets you write code against the general type and have it work for every specific one.

```java
Animal a = new Dog();
a.speak();          // runs Dog's speak()
a = new Cat();
a.speak();          // runs Cat's speak()
// same call, different behaviour at runtime
```



One superclass reference; the real object decides which method runs.

Encapsulation

Encapsulation keeps fields `private` and exposes controlled access through methods (getters and setters). This protects invariants — nothing can put the object in an invalid state.

```java
public class Account {
    private double balance;             // hidden
    public double getBalance() { return balance; }
    public void deposit(double amt) {
        if (amt > 0) balance += amt;    // guarded
    }
}
```

Abstraction

Abstraction exposes essential behaviour while hiding implementation. Abstract classes and interfaces let callers depend on what a type does, not how.

```java
abstract class Shape {
    abstract double area();   // what, not how
}
class Circle extends Shape {
    double r;
    double area() { return Math.PI * r * r; }
}
```

Pillar	In one line
Encapsulation	hide and guard internal state
Inheritance	reuse code via is-a
Polymorphism	one interface, many forms
Abstraction	expose essentials, hide detail

Design with the pillars

Encapsulate state, inherit to reuse, use polymorphism to write general code, and abstract to hide complexity. Applied together, they turn tangled logic into clean, maintainable objects.

© 2026 ByteMart Digital

MODULE 3 · CORE LANGUAGE FEATURES

CHAPTER 09

Interfaces & Abstract Classes

Defining contracts with interfaces, sharing partial code with abstract classes, and choosing between them.

Interfaces and abstract classes both let you program to an abstraction rather than a concrete type. An **interface** is a pure contract of behaviour; an **abstract class** is a partially-built base other classes complete. Knowing when to use each is a hallmark of good Java design.

Interfaces

An interface declares methods a type must provide. A class `implements` one (or many) and supplies the bodies. Since Java 8, interfaces can also have `default` and `static` methods.

java

```
interface Payable {
    double calculatePay();              // abstract
    default void report() {             // default method
        System.out.println("Pay: " + calculatePay());
    }
}

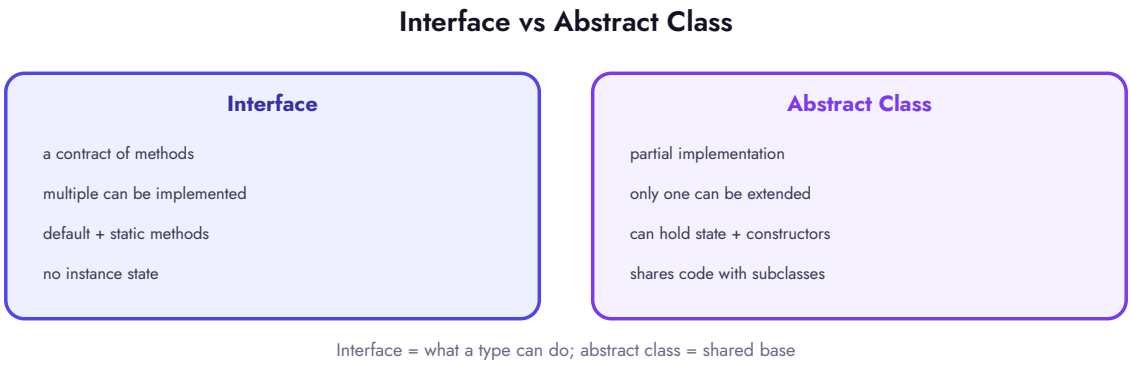
class Employee implements Payable {
    public double calculatePay() { return 50000; }
}
```

Abstract classes

An **abstract class** cannot be instantiated. It can hold state, constructors, and both concrete and abstract methods — ideal for sharing code among related subclasses.

java

```
abstract class Shape {
    private String name;                // shared state
    Shape(String name) { this.name = name; }
    abstract double area();             // subclasses must implement
    void describe() {                   // shared behaviour
        System.out.println(name + ": " + area());
    }
}
```



Interface defines what a type can do; an abstract class shares a base implementation.

Choosing between them

- Use an **interface** for a capability many unrelated types can have (`Comparable`, `Runnable`).
- Use an **abstract class** to share code and state among closely related types.
- A class can implement **many interfaces** but extend **one** class.
- Favour interfaces — they keep designs flexible and testable.

Multiple interfaces

java

```
class Duck implements Swimmer, Flyer {
    public void swim() { /* ... */ }
    public void fly()  { /* ... */ }
}
```

WATCH OUT

Since interfaces can have `default` methods, a class implementing two interfaces with the same default method must override it to resolve the conflict.

Contract vs shared base

Interfaces describe what a type can do and enable multiple inheritance of behaviour; abstract classes share real code among relatives. Reach for interfaces first, abstract classes when you need shared state.

MODULE 3 · CORE LANGUAGE FEATURES

CHAPTER 10

Packages & Access Modifiers

Organising code into packages and controlling visibility with the four access levels.

As programs grow, **packages** group related classes into namespaces, and **access modifiers** control who can see each member. Together they keep large codebases organised and enforce encapsulation across class boundaries.

Packages

A package is a namespace declared at the top of a file. It maps to a folder structure and prevents name clashes. You bring in other packages' classes with `import`.

```java
package com.bytemart.billing;   // this class's package

import java.util.List;          // use a class from another package
import com.bytemart.model.Invoice;
```

The four access levels

Modifier	Visible to
public	everyone
protected	same package + subclasses
(default)	same package only
private	same class only

'Default' access (no modifier) is also called **package-private** — visible only within the same package.

Applying them

```java
public class BankAccount {
    private double balance;          // only this class
    protected String owner;          // subclasses + package
    public String getOwner() {       // everyone
        return owner;
    }
}
```

Encapsulation across boundaries

- Make fields **private** by default; expose behaviour, not data.

- Use **public** only for the intended API of a class.
- Use **protected** to let subclasses extend behaviour.
- Keep helper classes **package-private** to hide them from the outside.

TIP

Reverse-domain package names (`com.bytemart.billing`) keep your packages globally unique and make the project structure self-documenting.

Namespace and visibility

Packages organise code and prevent clashes; access modifiers decide who sees what. Defaulting to `private` and exposing a small `public` surface is encapsulation applied at scale.

MODULE 3 · CORE LANGUAGE FEATURES

CHAPTER 11

Static, Final & Nested Classes

Class-level members with static, immutability with final, and grouping types with nested classes.

Three keywords shape how members belong and behave. `static` ties a member to the class rather than an instance; `final` prevents change; and nested classes group closely-related types together.

static members

A `static` field or method belongs to the class itself, shared across all instances and callable without an object. Constants and utility methods are the classic uses.

```java
public class MathUtil {
    public static final double PI = 3.14159;   // constant
    public static int square(int n) {          // utility
        return n * n;
    }
}
int r = MathUtil.square(5);   // no object needed
```

final

final on	Means
variable	cannot be reassigned
method	cannot be overridden
class	cannot be extended

```java
final int MAX = 100;         // constant value
final class Money { }        // no subclasses (like String)
```

Nested classes

A class defined inside another groups tightly-coupled types. A **static nested class** stands alone; an **inner (non-static) class** holds a reference to its enclosing instance.

```java
public class Outer {
    static class Builder { /* helper, no Outer needed */ }
    class Inner { /* has access to Outer's fields */ }
}
```

© 2026 ByteMart Digital

Anonymous classes

An **anonymous class** defines and instantiates a one-off implementation inline — historically used for listeners and callbacks, now often replaced by lambdas.

java

```
Runnable r = new Runnable() {
    public void run() { System.out.println("run"); }
};
// Modern equivalent: Runnable r = () -> System.out.println("run");
```

TIP

Combine `static final` for constants, and prefer `static` nested classes over inner classes unless you truly need the outer instance — they are lighter and avoid subtle memory leaks.

Belonging and immutability

`static` attaches members to the class, `final` locks things down, and nested classes keep related types together. Used well, they make intent explicit and code safer.

MODULE 3 · CORE LANGUAGE FEATURES

CHAPTER 12

Enums & Records

Fixed sets of constants with enums and concise immutable data carriers with records.

Two special class types make common patterns concise. An **enum** defines a fixed set of named constants safely; a **record** (Java 16+) is a compact, immutable data carrier that removes reams of boilerplate.

Enums

An `enum` is a type whose values are a fixed set of constants. It is type-safe — you cannot assign an invalid value — and can hold fields and methods like any class.

```java
enum Status { ACTIVE, PAUSED, CLOSED }

Status s = Status.ACTIVE;
switch (s) {
    case ACTIVE -> System.out.println("running");
    case PAUSED -> System.out.println("on hold");
    case CLOSED -> System.out.println("done");
}
```

Enums with data

```java
enum Planet {
    EARTH(9.8), MARS(3.7);
    private final double gravity;
    Planet(double g) { this.gravity = g; }
    double gravity() { return gravity; }
}
System.out.println(Planet.MARS.gravity());   // 3.7
```

Records

A **record** declares an immutable data class in one line. Java generates the constructor, accessors, `equals`, `hashCode`, and `toString` automatically.

```java
record Point(int x, int y) { }

Point p = new Point(3, 4);
System.out.println(p.x());        // 3  (auto accessor)
System.out.println(p);            // Point[x=3, y=4]
System.out.println(p.equals(new Point(3, 4)));   // true
```

OUTPUT

```
3
Point[x=3, y=4]
true
```

When to use each

- **Enum** — a fixed, known set of values (states, roles, directions).
- **Record** — a simple immutable bundle of data (DTOs, coordinates, results).
- Records are **final** and their fields are **immutable** by design.
- Both reduce boilerplate and make intent obvious.

Concise, safe types

Enums give you type-safe constant sets; records give you immutable data classes for free. Both replace verbose, error-prone hand-written code with clear, intentional declarations.

© 2026 ByteMart Digital

MODULE 4 · ESSENTIAL APIS

CHAPTER 13

Strings & Text

Working with immutable Strings, the string pool, and efficient building with StringBuilder.

Strings are among the most-used types in Java, and they behave in some important ways. Strings are **immutable**, share a special pool, and have a large API. Understanding immutability and when to use `StringBuilder` avoids both bugs and performance traps.

Strings are immutable

Once created, a String never changes. Every operation that seems to modify it — `concat`, `replace`, `toUpperCase` — returns a **new** String and leaves the original untouched.

java

```
String s = "hello";
s.toUpperCase();          // returns a new String, ignored
System.out.println(s);    // hello  (unchanged)
s = s.toUpperCase();      // reassign to keep the result
System.out.println(s);    // HELLO
```

OUTPUT

```
hello
HELLO
```

The string pool

String literals are interned in a shared **pool**, so identical literals reference the same object. This is why `==` sometimes appears to work on strings — but you should always compare content with `.equals()`.

java

```
String a = "hi";
String b = "hi";
System.out.println(a == b);        // true  (same pooled object)
String c = new String("hi");
System.out.println(a == c);        // false (new object)
System.out.println(a.equals(c));   // true  (same content)
```

Common String methods

- `length()`, `charAt(i)`, `substring(a, b)`
- `indexOf`, `contains`, `startsWith`, `endsWith`
- `trim` / `strip`, `split`, `replace`, `toUpperCase`
- `String.format(...)` and text blocks for multi-line text

StringBuilder for heavy work

Building a string in a loop with + creates many throwaway objects. `StringBuilder` is a mutable buffer that is far more efficient for repeated concatenation.

java

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) sb.append(i).append(",");
System.out.println(sb.toString());   // 0,1,2,3,4,
```

WATCH OUT

Never build strings with + inside a large loop — each iteration allocates a new String. Use `StringBuilder` for anything performance-sensitive.

Immutable, pooled, buildable

Strings never change, share a pool, and are compared by content with `.equals()`. For repeated concatenation, reach for `StringBuilder` to avoid needless allocation.

MODULE 4 · ESSENTIAL APIS

CHAPTER 14

Arrays

Fixed-size ordered collections — declaring, iterating, and their key limitations.

An array is a fixed-size, indexed container of elements of one type. Arrays are fast and simple, but their size is fixed at creation — which is why the Collections framework (next module) usually takes over for dynamic data.

Declaring and creating

java

```
int[] nums = new int[5];        // five zeros
int[] primes = {2, 3, 5, 7, 11}; // literal
String[] names = new String[3];  // three nulls

nums[0] = 42;
System.out.println(nums.length); // 5  (fixed size)
```

Iterating

java

```
for (int i = 0; i < primes.length; i++)
    System.out.print(primes[i] + " ");

for (int p : primes)            // for-each: cleaner
    System.out.print(p + " ");
```

Multidimensional arrays

Java arrays can nest. A 2D array is really an array of arrays, which allows 'jagged' rows of different lengths.

java

```
int[][] grid = {
    {1, 2, 3},
    {4, 5, 6},
};
System.out.println(grid[1][2]);   // 6
```

The Arrays utility class

- `Arrays.sort(a)` — sort in place.
- `Arrays.toString(a)` — readable printout.
- `Arrays.copyOf(a, n)` — resize into a new array.
- `Arrays.asList(a)` — view as a List.

© 2026 ByteMart Digital

```java
int[] a = {3, 1, 2};
Arrays.sort(a);
System.out.println(Arrays.toString(a));   // [1, 2, 3]
```

WATCH OUT

Array size is fixed at creation — there is no `add` or `remove`. Accessing an index outside the bounds throws `ArrayIndexOutOfBoundsException`. For growable data, use a `List`.

Fast but fixed

Arrays give quick, indexed access to a fixed number of elements. When you need to grow, shrink, or search flexibly, the Collections framework is almost always the better tool.

MODULE 4 · ESSENTIAL APIS

CHAPTER 15

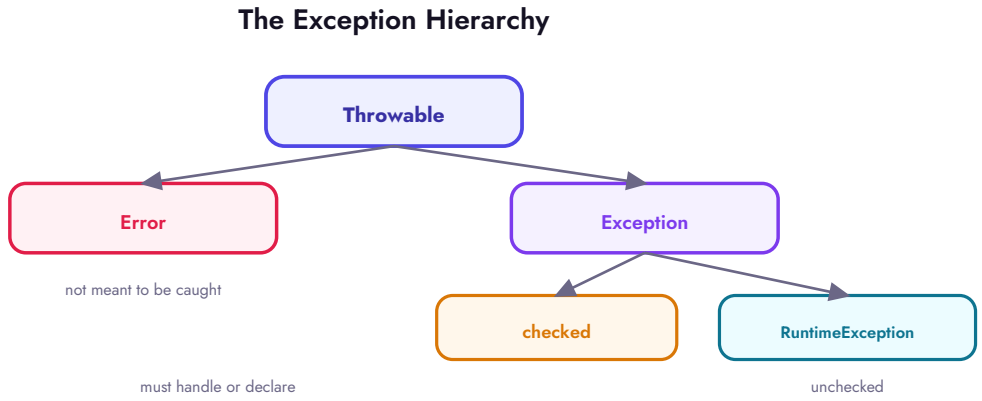
Exceptions & Error Handling

Handling failures with try-catch, the checked vs unchecked distinction, and clean resource management.

Exceptions are Java's mechanism for handling errors. When something goes wrong, an exception is **thrown** and propagates up the call stack until it is **caught**. Handling them well makes the difference between a robust program and one that crashes on the first surprise.

The hierarchy

All errors descend from `Throwable`. **Errors** are serious problems you usually cannot recover from; **Exceptions** are conditions your program can handle, split into checked and unchecked.



Throwable splits into Error and Exception; exceptions are checked or unchecked.

try-catch-finally

```java
try {
    int result = 10 / divisor;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {
    System.out.println("always runs");
}
```

Checked vs unchecked

Kind	Rule
Checked	must be caught or declared with throws
Unchecked	runtime; not required to handle
Error	serious; not meant to be caught

`IOException` is checked — the compiler forces you to handle it. `NullPointerException` is unchecked — it signals a programming bug.

Throwing and custom exceptions

```java
public void withdraw(double amt) {
    if (amt > balance)
        throw new IllegalArgumentException("Insufficient funds");
    balance -= amt;
}

class PaymentException extends Exception { }   // custom checked
```

try-with-resources

Resources like files and streams must be closed. **try-with-resources** closes them automatically — even if an exception is thrown — for any type implementing `AutoCloseable`.

```java
try (var reader = new BufferedReader(new FileReader("data.txt"))) {
    System.out.println(reader.readLine());
}   // reader is closed automatically
```

WATCH OUT

Never swallow an exception with an empty `catch` block — it hides bugs. At minimum log it, and only catch what you can meaningfully handle.

Handle, don't hide

Exceptions separate error handling from normal logic. Catch what you can handle, let the rest propagate, close resources with try-with-resources, and never silently swallow failures.

© 2026 ByteMart Digital

MODULE 4 · ESSENTIAL APIS

CHAPTER 16

Generics

Type-safe, reusable code with generic classes and methods — and how the compiler enforces it.

Generics let you write classes and methods that work with any type while keeping full type safety. Instead of casting objects and hoping, the compiler checks types for you — the reason `List<String>` is so much safer than a raw list.

> **Generics**
> A feature that parameterises types, letting classes and methods operate on a type supplied by the caller, with the compiler enforcing type correctness.

Why generics

```java
// Without generics: unsafe, needs casts
List raw = new ArrayList();
raw.add("hi");
String s = (String) raw.get(0);   // cast, can fail at runtime

// With generics: safe, no cast
List<String> list = new ArrayList<>();
list.add("hi");
String t = list.get(0);           // compiler guarantees String
```

Generic classes

```java
class Box<T> {
    private T value;
    void set(T value) { this.value = value; }
    T get() { return value; }
}

Box<Integer> b = new Box<>();
b.set(42);
int n = b.get();   // no cast needed
```

Generic methods

```java
static <T> T firstOf(List<T> list) {
    return list.get(0);
}
String first = firstOf(List.of("a", "b"));
```

© 2026 ByteMart Digital

Bounded types and wildcards

- `<T extends Number>` — T must be a Number or subtype.
- `<?>` — an unknown type (read-only use).
- `<? extends T>` — T or any subtype (a producer).
- `<? super T>` — T or any supertype (a consumer).

java

```
static double sum(List<? extends Number> nums) {
    double total = 0;
    for (Number n : nums) total += n.doubleValue();
    return total;
}
```

NOTE

Generics are enforced at compile time and then **erased** at runtime (type erasure). This is why you cannot write `new T[]` or check `instanceof List<String>`.

Reusable and type-safe

Generics let one class or method serve many types while the compiler guarantees correctness. They are the backbone of the Collections framework and the reason modern Java rarely needs casts.

© 2026 ByteMart Digital

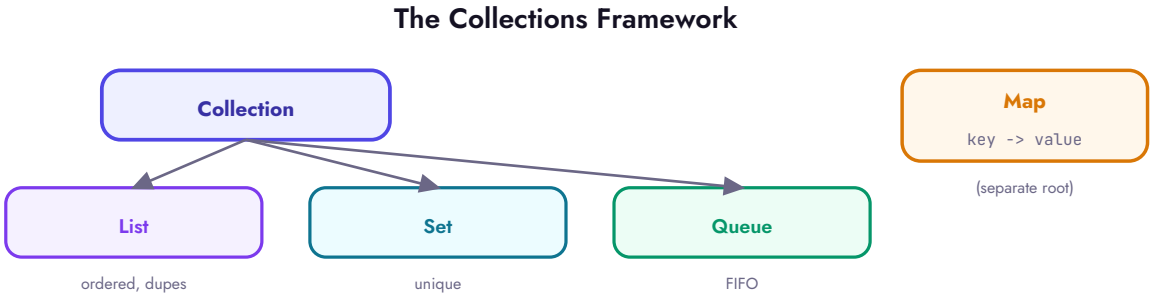
MODULE 5 · COLLECTIONS & STREAMS

CHAPTER 17

The Collections Framework

Java's core data structures — List, Set, and Map — and how to choose between them.

The Collections framework provides ready-made, growable data structures that replace fixed-size arrays for almost all real work. Its three pillars — **List**, **Set**, and **Map** — cover ordered sequences, unique elements, and key-value pairs.



List, Set, and Queue extend Collection; Map is a separate root.

List — ordered, allows duplicates

```java
List<String> names = new ArrayList<>();
names.add("Asha");
names.add("Ravi");
names.get(0);                 // Asha
names.remove("Ravi");
for (String n : names) System.out.println(n);
```

Set — unique elements

```java
Set<Integer> ids = new HashSet<>();
ids.add(1); ids.add(1); ids.add(2);
System.out.println(ids.size());   // 2  (duplicate ignored)
```

Map — key to value

```java
Map<String, Integer> stock = new HashMap<>();
stock.put("apple", 5);
stock.put("pear", 3);
System.out.println(stock.get("apple"));          // 5
stock.getOrDefault("mango", 0);                  // 0
stock.forEach((k, v) -> System.out.println(k + "=" + v));
```

Choosing an implementation

Need	Use
indexed, ordered list	ArrayList
frequent insert/remove ends	LinkedList / ArrayDeque
unique, fast lookup	HashSet
sorted unique	TreeSet
key-value, fast lookup	HashMap
sorted keys	TreeMap

TIP

Program to the interface: declare variables as `List`, `Set`, or `Map`, not the concrete class. You can then swap implementations without touching the rest of your code.

The right structure

List for ordered sequences, Set for uniqueness, Map for lookups. Picking the correct collection — and coding to its interface — is one of the highest-leverage decisions in everyday Java.

© 2026 ByteMart Digital

MODULE 5 · COLLECTIONS & STREAMS

CHAPTER 18

Iterators & Comparators

Traversing collections safely and defining custom sort orders with Comparator.

Two utilities make collections flexible: **iterators** traverse elements one at a time (and remove safely), and **comparators** define how elements should be ordered. Together they power sorting and controlled iteration.

Iterating and removing safely

Removing from a collection while looping over it with a for-each throws a `ConcurrentModificationException`. An `Iterator`'s own `remove()` is the safe way.

```java
Iterator<Integer> it = numbers.iterator();
while (it.hasNext()) {
    if (it.next() < 0) it.remove();   // safe removal
}
// Or simply: numbers.removeIf(n -> n < 0);
```

Natural ordering: Comparable

A class implements `Comparable` to define its **natural order** — how `Collections.sort` orders it by default.

```java
class Person implements Comparable<Person> {
    int age;
    public int compareTo(Person other) {
        return Integer.compare(age, other.age);   // by age
    }
}
```

Custom ordering: Comparator

A `Comparator` defines an alternative order without changing the class. Modern comparators are built fluently with `comparing` and `thenComparing`.

```java
people.sort(Comparator.comparing(Person::getName));

people.sort(
    Comparator.comparingInt(Person::getAge)
              .thenComparing(Person::getName)   // tie-breaker
              .reversed());
```

Comparable vs Comparator

	Comparable	Comparator
Defines	natural order	custom order
Where	inside the class	outside, separate
Count	one	many

TIP

Use `Comparator.comparing(...)` with method references for clean, chainable sorting. It reads almost like plain English and avoids error-prone hand-written comparisons.

Traverse and order

Iterators let you walk and safely modify a collection; comparators let you order it any way you need. The fluent `Comparator` API makes even multi-key sorting concise and readable.

MODULE 5 · COLLECTIONS & STREAMS

CHAPTER 19

Lambdas & Functional Interfaces

Passing behaviour as data with lambda expressions and the functional interfaces they implement.

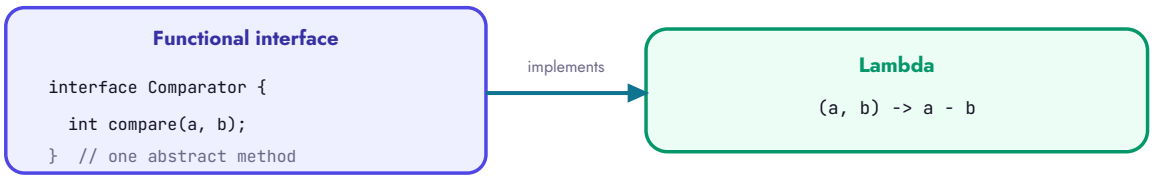
Since Java 8, you can treat behaviour as data. A **lambda** is a concise anonymous function; it implements a **functional interface** — any interface with a single abstract method. This unlocks a more expressive, functional style, and it is what streams are built on.

> **Functional interface**
> An interface with exactly one abstract method, which a lambda expression can implement inline.

Lambda syntax

```java
// (parameters) -> body
Runnable r = () -> System.out.println("run");
Comparator<String> byLen = (a, b) -> a.length() - b.length();
Function<Integer, Integer> square = n -> n * n;
```

Lambdas Implement Functional Interfaces



A lambda is a concise instance of a single-method interface

A lambda is a concise instance of a single-method (functional) interface.

The built-in functional interfaces

Interface	Shape
Function	T in, R out
Predicate	T in, boolean out
Consumer	T in, nothing out
Supplier	nothing in, T out
BiFunction	two in, R out

```java
Predicate<Integer> isEven = n -> n % 2 == 0;
System.out.println(isEven.test(4));   // true

Consumer<String> print = System.out::println;
print.accept("hi");
```

Method references

When a lambda just calls an existing method, a **method reference** is shorter and clearer. `String::toUpperCase` replaces `s -> s.toUpperCase()`.

```java
names.forEach(System.out::println);       // instance method
names.stream().map(String::toUpperCase);  // ...on each element
List<Person> p = ids.stream().map(Person::new).toList(); // constructor
```

TIP

Reach for a method reference whenever a lambda's body is a single existing call — it removes noise and states intent directly.

Behaviour as a value

Lambdas turn behaviour into something you can pass, store, and compose, implementing single-method interfaces on the fly. They are the foundation of streams and modern, expressive Java.

MODULE 5 · COLLECTIONS & STREAMS

CHAPTER 20

The Streams API

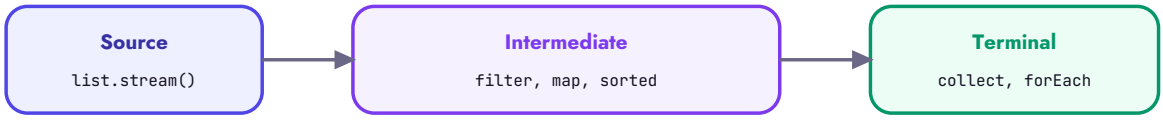
Processing collections declaratively with a pipeline of filter, map, and reduce operations.

The Streams API lets you process collections declaratively — describing what you want, not how to loop. A stream is a pipeline of operations over a data source, replacing verbose loops with clear, composable transformations.

The pipeline

A stream has three parts: a **source**, zero or more **intermediate** operations (lazy), and one **terminal** operation that produces a result and triggers execution.

The Stream Pipeline



Intermediate ops are lazy; the terminal op triggers execution

```
list.stream().filter(...).map(...).collect(...)
```

A stream flows from a source through lazy intermediate ops to a terminal op.

A first pipeline

```java
List<String> names = List.of("Asha", "Ravi", "Meera", "Ken");

List<String> result = names.stream()
    .filter(n -> n.length() > 3)      // keep longer names
    .map(String::toUpperCase)         // transform
    .sorted()                         // order
    .toList();                        // terminal
// [ASHA, MEERA, RAVI]
```

Common operations

Operation	Does
filter(pred)	keep matching elements
map(fn)	transform each element
sorted()	order elements
distinct()	remove duplicates

Operation	Does
limit(n)	take first n
collect / toList	gather into a collection
count / sum / reduce	aggregate to one value

Reducing and collecting

```java
int total = nums.stream().mapToInt(Integer::intValue).sum();

Map<Boolean, List<Integer>> parts = nums.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));

String joined = names.stream().collect(Collectors.joining(", "));
```

Why streams

- **Declarative** — read the intent, not the loop mechanics.
- **Composable** — chain operations into a clear pipeline.
- **Lazy** — intermediate ops run only when a terminal op demands.
- **Parallelisable** — `parallelStream()` for big data (use with care).

WATCH OUT

Streams are single-use — once a terminal operation runs, the stream is consumed. Do not reuse a stream; create a new one from the source instead.

Describe the transformation

Streams replace manual loops with a readable pipeline of filter, map, and reduce. Master them and most collection-processing code becomes shorter, clearer, and easier to reason about.

© 2026 ByteMart Digital

MODULE 6 · MODERN & CONCURRENT JAVA

CHAPTER 21

Optional & Null Safety

Replacing error-prone nulls with Optional to express and handle absent values safely.

The `NullPointerException` is Java's most common runtime error. `Optional` is a container that explicitly represents 'a value or nothing', forcing callers to consider the empty case and making absence part of the type.

`Optional`
A container object that may or may not hold a non-null value, used to model potentially-absent results without returning null.

Creating and using Optional

```java
Optional<User> found = repository.findById(id);

if (found.isPresent()) {
    System.out.println(found.get().getName());
}

// Better: functional style, no explicit check
found.map(User::getName)
     .ifPresent(System.out::println);
```

Providing defaults

```java
String name = found.map(User::getName)
                   .orElse("Guest");            // default value

User user = found.orElseThrow(
    () -> new NotFoundException(id));            // or throw
```

Returning Optional

Methods that may not find a result should return `Optional<T>` rather than `null`. The signature then tells callers absence is possible, and the compiler nudges them to handle it.

```java
public Optional<Product> findByName(String name) {
    return products.stream()
                   .filter(p -> p.getName().equals(name))
                   .findFirst();   // returns Optional
}
```

Good Optional habits

- Use it as a **return type** for 'maybe a value' methods.
- Prefer `map`, `filter`, `orElse` over `isPresent` + `get`.
- Do **not** use Optional for fields or method parameters.
- Never call `get()` without checking — it can throw.

WATCH OUT

`Optional` is not a cure-all: using it for fields or parameters adds overhead and noise. Its sweet spot is expressing an absent return value.

Make absence explicit

Optional turns 'might be null' from a hidden landmine into a visible part of the type. Used for return values and handled functionally, it eliminates a large share of null bugs.

© 2026 ByteMart Digital

MODULE 6 · MODERN & CONCURRENT JAVA

CHAPTER 22

Concurrency & Threads

Running work in parallel with threads, and the shared-state hazards you must guard against.

Concurrency lets a program do several things at once — essential for responsive apps and busy servers. Java's unit of concurrent work is the **thread**. Power comes with peril: shared mutable state is where most concurrency bugs live.

Creating threads

The modern way is to pass a `Runnable` (often a lambda) to a thread or, better, to an executor. Directly subclassing `Thread` is discouraged.

```java
Runnable task = () -> System.out.println("working on " +
    Thread.currentThread().getName());

Thread t = new Thread(task);
t.start();      // runs concurrently
t.join();       // wait for it to finish
```

The shared-state problem

When two threads modify the same data without coordination, updates can be lost — a **race condition**. The classic example is two threads incrementing the same counter.

```java
// Not thread-safe: count++ is read-modify-write
class Counter {
    int count;
    void inc() { count++; }   // two threads can lose updates
}
```

synchronized

The `synchronized` keyword ensures only one thread at a time runs a block or method for a given lock, preventing interleaved updates.

```java
class SafeCounter {
    private int count;
    synchronized void inc() { count++; }   // mutually exclusive
    synchronized int get() { return count; }
}
```

Key concurrency terms

- **Race condition** — result depends on unlucky thread timing.
- **Deadlock** — threads wait on each other forever.
- **Atomicity** — an operation completes as one indivisible step.
- **Visibility** — one thread's writes becoming visible to another (see `volatile`).

WATCH OUT

Concurrency bugs are intermittent and hard to reproduce. Minimise shared mutable state, and prefer the higher-level utilities in the next chapter over hand-managed locks.

Parallel, but guarded

Threads unlock parallelism, but shared mutable state must be coordinated or it corrupts. Guard it with `synchronized` — and lean on the concurrency utilities that make safety easier.

© 2026 ByteMart Digital

MODULE 6 · MODERN & CONCURRENT JAVA

CHAPTER 23

Concurrency Utilities

Higher-level tools — executors, futures, and concurrent collections — that make parallelism safer.

Managing raw threads and locks by hand is error-prone. The `java.util.concurrent` package provides higher-level building blocks — thread pools, futures, atomic variables, and concurrent collections — that handle the hard parts for you.

Executors and thread pools

An **executor** manages a pool of reusable threads, so you submit tasks instead of creating threads yourself. This is the recommended way to run concurrent work.

```java
ExecutorService pool = Executors.newFixedThreadPool(4);
pool.submit(() -> doWork());
pool.shutdown();   // stop accepting; finish queued tasks
```

Futures for results

A task that returns a value gives you a `Future` — a handle to a result that will be ready later. `CompletableFuture` adds a fluent way to chain async steps.

```java
Future<Integer> f = pool.submit(() -> compute());
Integer result = f.get();   // blocks until ready

CompletableFuture.supplyAsync(this::load)
    .thenApply(this::transform)
    .thenAccept(System.out::println);
```

Atomic variables

`AtomicInteger` and friends provide lock-free, thread-safe updates for single variables — simpler and faster than `synchronized` for counters.

```java
AtomicInteger count = new AtomicInteger();
count.incrementAndGet();   // safe without locks
```

Concurrent collections

Instead of	Use
HashMap (shared)	ConcurrentHashMap

Instead of	Use
ArrayList (shared)	CopyOnWriteArrayList
manual queue	BlockingQueue

TIP

Default to executors and concurrent collections rather than raw threads and `synchronized`. They are tested, efficient, and eliminate whole categories of concurrency bugs.

Let the library do it

The concurrency utilities package raw threading into safe, reusable tools. Executors run your tasks, futures deliver results, and concurrent collections handle shared state — use them instead of rolling your own.

© 2026 ByteMart Digital

MODULE 6 · MODERN & CONCURRENT JAVA

CHAPTER 24

Files & I/O

Reading and writing files and streams with the modern NIO API.

Programs constantly read and write data — files, network streams, the console. Java's I/O splits into the classic stream-based API and the modern **NIO** (`java.nio.file`), which offers simpler, more powerful file handling.

Reading a file the modern way

java

```
import java.nio.file.*;

Path path = Path.of("data.txt");
List<String> lines = Files.readAllLines(path);   // whole file
String content = Files.readString(path);         // as one String

lines.forEach(System.out::println);
```

Writing a file

java

```
Files.writeString(path, "Hello ByteMart\n");
Files.write(Path.of("log.txt"),
    List.of("line 1", "line 2"),
    StandardOpenOption.CREATE, StandardOpenOption.APPEND);
```

Streaming large files

For big files, stream lines lazily instead of loading everything into memory. Wrap it in try-with-resources so the file handle is always closed.

java

```
try (Stream<String> lines = Files.lines(path)) {
    long errors = lines.filter(l -> l.contains("ERROR")).count();
    System.out.println(errors);
}
```

Streams vs readers

- **Byte streams** (`InputStream`/`OutputStream`) — raw binary data.
- **Character streams** (`Reader`/`Writer`) — text with an encoding.
- **Buffered** wrappers — reduce system calls for speed.
- **NIO Files** — concise helpers for common file tasks.

© 2026 ByteMart Digital

WATCH OUT

Always close streams and readers — leaked file handles exhaust system resources. Use try-with-resources so closing is automatic even when an exception is thrown.

Modern, safe I/O

The NIO `Files` API turns most file work into one-liners, while streaming and try-with-resources keep it memory-safe and leak-free. Reach for `java.nio.file` first.

© 2026 ByteMart Digital

MODULE 7 · JVM & MEMORY

CHAPTER 25

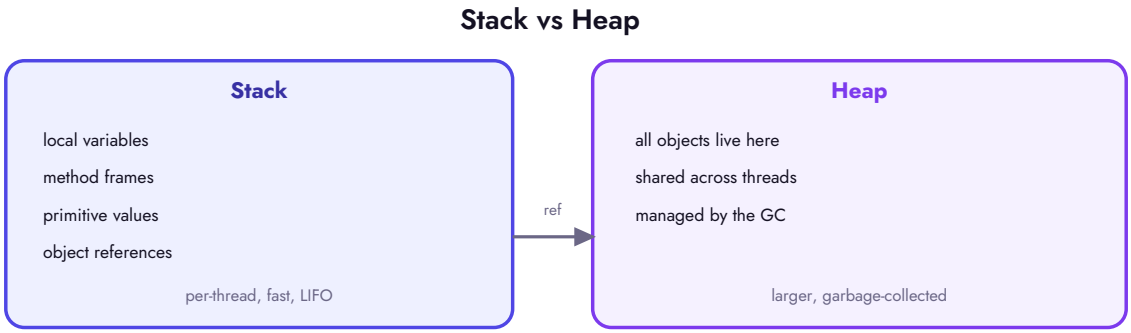
Memory & Garbage Collection

How the JVM manages memory across the stack and heap, and how garbage collection frees objects.

Java manages memory for you — but understanding how helps you write efficient code and diagnose leaks. Memory divides into the **stack** (per-thread, for calls and locals) and the **heap** (shared, for objects), with the **garbage collector** reclaiming what is no longer used.

Stack vs heap

Each thread has a **stack** holding method frames, local variables, and object references. All **objects** live on the shared **heap**. A local variable holds a reference on the stack that points to an object on the heap.



Locals and references live on the stack; all objects live on the heap.

Garbage collection

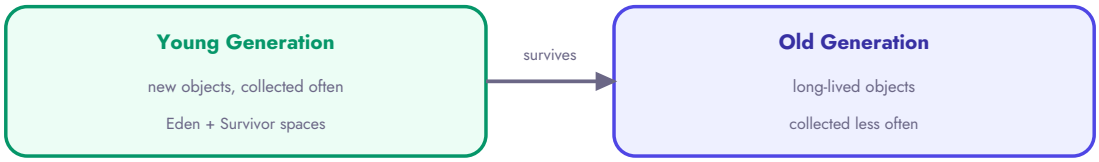
The **garbage collector** automatically frees heap objects that are no longer reachable from any live reference. You never call `free()` — when nothing points to an object, it becomes eligible for collection.

```java
User u = new User();   // object on heap, u references it
u = null;              // no references left
// the User object is now eligible for garbage collection
```

Generational GC

Most objects die young, so the heap is split into generations. New objects go in the **young generation** (collected often and cheaply); survivors are promoted to the **old generation** (collected rarely).

Garbage Collection Generations



Objects that survive young-gen GC are promoted to old-gen

Objects surviving young-generation GC are promoted to the old generation.

Memory leaks in a managed language

- Objects still **referenced** but never used — e.g. forgotten entries in a static collection.
- Unclosed resources holding native memory.
- Listeners or caches that are never removed.
- The fix: drop references you no longer need.

WATCH OUT

'Managed memory' does not mean 'leak-free'. If a live object keeps referencing data you no longer need, the GC cannot reclaim it. Clear references and close resources.

References decide lifetime

Objects live as long as something reachable references them. The stack holds references, the heap holds objects, and generational GC reclaims the unreferenced — so managing references is managing memory.

© 2026 ByteMart Digital

MODULE 7 · JVM & MEMORY

CHAPTER 26

equals, hashCode & Object Contracts

Correctly overriding equals and hashCode so objects behave properly in collections.

Every class inherits `equals`, `hashCode`, and `toString` from `Object`. Their default behaviour is identity-based, which is often wrong for data objects. Overriding `equals` and `hashCode` correctly is essential for objects used in sets and maps.

The default is identity

By default, `equals` returns true only for the same object, and `hashCode` is derived from the object's address. Two 'equal' data objects will look different unless you override both.

Overriding equals

java

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Point p)) return false;
    return x == p.x && y == p.y;
}
```

Overriding hashCode

If two objects are equal, they **must** have the same hash code — or hash-based collections will misbehave. Use `Objects.hash` with the same fields you used in `equals`.

java

```
@Override
public int hashCode() {
    return Objects.hash(x, y);   // same fields as equals
}
```

The equals/hashCode contract

- Equal objects must return the **same** hashCode.
- `equals` must be reflexive, symmetric, transitive, and consistent.
- Use the **same fields** in both methods.
- Unequal objects may share a hashCode (a collision) — that is allowed.

WATCH OUT

Overriding `equals` without `hashCode` breaks `HashMap` and `HashSet`: two equal objects can both be stored, or a lookup can fail to find one. Always override them together.

© 2026 ByteMart Digital

NOTE

A **record** generates a correct `equals`, `hashCode`, and `toString` for you — often the simplest way to get these right for data classes.

Equal means equal everywhere

For objects compared by value, override `equals` and `hashCode` together, over the same fields. Get this right and your objects behave correctly in every collection; get it wrong and lookups silently fail.

MODULE 7 · JVM & MEMORY

CHAPTER 27

Annotations & Reflection

Adding metadata with annotations and inspecting or manipulating code at runtime with reflection.

Annotations attach metadata to code; **reflection** inspects and manipulates classes at runtime. Together they power much of modern Java — frameworks like Spring and JUnit use them to wire and drive your code without you writing glue.

Annotations

An annotation is a marker read by the compiler, tools, or frameworks. You have already met `@Override`; libraries define many more, and you can write your own.

```java
@Override            // compiler check
@Deprecated          // signals 'do not use'
@FunctionalInterface // enforces one abstract method

// Framework annotations you'll see often:
@Entity @Test @Autowired @RestController
```

Defining an annotation

```java
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Timed { }

class Service {
    @Timed void process() { /* ... */ }
}
```

Reflection

Reflection lets code examine classes, methods, and fields — and invoke them — at runtime. It is how frameworks discover your annotated methods and call them.

```java
Class<?> clazz = Service.class;
for (Method m : clazz.getDeclaredMethods()) {
    if (m.isAnnotationPresent(Timed.class)) {
        System.out.println("timed: " + m.getName());
    }
}
```

Where you meet them

© 2026 ByteMart Digital

- **Spring** — `@Component`, `@Autowired` wire beans via reflection.
- **JUnit** — `@Test` methods are found and run reflectively.
- **JPA/Hibernate** — `@Entity`, `@Id` map objects to tables.
- **Jackson** — annotations control JSON serialisation.

WATCH OUT

Reflection is powerful but slow and it bypasses compile-time checks. Use it for frameworks and tooling — not for everyday application logic where normal calls are clearer and safer.

Metadata that drives frameworks

Annotations describe intent; reflection acts on it at runtime. This pairing is the engine behind Spring, JUnit, and JPA — which is why understanding it demystifies how those frameworks 'know' what to do.

© 2026 ByteMart Digital

MODULE 8 · PROFESSIONAL JAVA

CHAPTER 28

Build Tools & the Ecosystem

Managing dependencies and builds with Maven and Gradle, and packaging your app as a JAR.

Real projects depend on libraries and need a repeatable build. **Build tools** — Maven and Gradle — manage dependencies, compile, test, and package your application, replacing fragile hand-run commands with a declarative, shareable configuration.

Why a build tool

- **Dependency management** — declare libraries; the tool fetches them and their dependencies.
- **Standard build** — compile, test, and package with one command.
- **Reproducible** — the same build on every machine and CI server.
- **Plugins** — extend the build for docs, coverage, packaging, and more.

Maven

Maven uses an XML `pom.xml` and a fixed project layout. You declare dependencies, and Maven resolves them from central repositories.

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <version>3.2.0</version>
</dependency>
```

```bash
mvn clean package     # compile, test, build the JAR
mvn test              # run tests
```

Gradle

Gradle uses a concise Groovy or Kotlin script instead of XML and is known for fast, incremental builds. It is the default for Android and increasingly common on the server.

```text
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
}
```

JARs and packaging

A **JAR** (Java ARchive) bundles compiled classes and resources into one distributable file. An **executable/fat JAR** also includes dependencies so it runs standalone with `java -jar app.jar`.

TIP

For a new project, generate the scaffold with a tool like Spring Initializr or `mvn archetype`. Starting from a standard layout saves time and keeps your build conventional.

Declare, then build

Build tools turn dependencies and packaging into declarative configuration anyone can reproduce. Maven and Gradle are the backbone of every serious Java project — learn one well.

© 2026 ByteMart Digital

MODULE 8 · PROFESSIONAL JAVA

CHAPTER 29

Testing with JUnit

Writing automated tests with JUnit — assertions, structure, and testing best practices.

Automated tests let you change code with confidence. **JUnit** is the standard Java testing framework: you write small methods that exercise your code and assert on the results, and the tooling runs them and reports failures.

A JUnit test

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {
    @Test
    void addsTwoNumbers() {
        Calculator calc = new Calculator();
        assertEquals(5, calc.add(2, 3));
    }
}
```

Common assertions

Assertion	Checks
assertEquals(a, b)	values are equal
assertTrue(cond)	condition is true
assertNull(x)	value is null
assertThrows(E, fn)	code throws E

```java
@Test
void rejectsNegative() {
    assertThrows(IllegalArgumentException.class,
        () -> account.withdraw(-5));
}
```

Arrange, Act, Assert

Structure each test in three clear phases: **arrange** the inputs, **act** by calling the code, and **assert** on the outcome. This keeps tests readable and focused on one behaviour.

Testing best practices

- One logical **behaviour** per test; name it for what it verifies.
- Tests must be **independent** and repeatable — no shared mutable state.
- Prioritise **critical paths** and past bugs over chasing 100% coverage.
- Use **mocks** (e.g. Mockito) to isolate the unit under test.

TIP

Write a test the moment you fix a bug — it documents the fix and stops the bug returning. A focused, fast suite is used far more than an exhaustive slow one.

Confidence to change

JUnit tests capture how your code should behave and catch regressions instantly. A well-structured suite — arrange, act, assert — is what lets you refactor and ship without fear.

© 2026 ByteMart Digital

MODULE 8 · PROFESSIONAL JAVA

CHAPTER 30

Best Practices & Clean Code

The habits and principles that make Java code readable, robust, and maintainable.

Everything in this guide comes together in how you write day-to-day code. Clean Java is not about clever tricks — it is about clear names, small methods, good encapsulation, and sensible defaults that make code easy for the next person (often future you) to read and change.

Naming and structure

- Classes are **nouns** (`InvoiceService`); methods are **verbs** (`sendReminder`).
- Keep methods **short** and doing one thing.
- Prefer clear names over comments that explain unclear code.
- One class, one responsibility.

Encapsulation and immutability

- Make fields **private**; expose behaviour, not raw data.
- Prefer **immutable** objects — use `final` fields and records.
- Return **Optional** for maybe-absent values, never leak nulls.
- Validate inputs at the boundary and fail fast with clear exceptions.

Collections and modern APIs

- Program to **interfaces** (`List`, `Map`), not implementations.
- Use **streams** for clear data transformations, plain loops when clearer.
- Choose the **right collection** for the access pattern.
- Override `equals` and `hashCode` together for value types.

Robustness

- Handle exceptions you can act on; never swallow them silently.
- Close resources with **try-with-resources**.
- Guard shared state in concurrent code; prefer the concurrency utilities.
- Write **tests** for critical logic and every bug you fix.

TIP

Adopt a consistent style and let tools enforce it — a formatter, static analysis (SpotBugs, SonarLint), and code review catch far more than willpower alone.

Clear beats clever

Great Java reads plainly: good names, small focused methods, tight encapsulation, and modern APIs used with judgement. Consistency and clarity — not cleverness — are what keep a codebase healthy as it grows.

© 2026 ByteMart Digital